

Tarea-04, Sistemas Operativos, Sistema de Archivos Simple en C

Estudiantes:

- Barrantes Jimenez Anthony
- Cabrera Tabash Samir
- Hernandez Cordoba Deyton

Descripción

Implementación de un sistema de archivos simulado que emula operaciones básicas como crear, escribir, leer, eliminar y listar archivos. El sistema utiliza una arquitectura modular con gestión de bloques de memoria y una tabla de archivos.

Características

- **Gestión de bloques:** Sistema de bloques de 512 bytes con asignación dinámica
- **Operaciones completas:** CREATE, WRITE, READ, DELETE, LIST
- **Manejo robusto de errores:** Validación exhaustiva de parámetros y operaciones
- **Código modular:** Separación clara de responsabilidades en módulos independientes
- **Interfaz interactiva:** CLI intuitiva para interactuar con el sistema

Arquitectura del Sistema

Estructura de Módulos

```
filesystem/  
├── filesystem.h/c      - API principal del sistema  
├── block_manager.h/c   - Gestión de bloques de memoria  
├── file_operations.h/c - Operaciones sobre archivos  
├── main.c              - Programa principal con CLI  
├── Makefile            - Sistema de compilación  
└── test_script.sh      - Pruebas automáticas
```

Especificaciones Técnicas

- **Tamaño de bloque:** 512 bytes
- **Bloques totales:** 2048 (1 MB de almacenamiento)
- **Máximo de archivos:** 100
- **Tamaño máximo por archivo:** 1 MB
- **Longitud máxima de nombre:** 256 caracteres

Compilación

Requisitos

- GCC 4.8 o superior
- Make
- Sistema operativo: Linux/Unix/macOS

Comandos de Compilación

```
# Compilar el proyecto
make

# Compilar y ejecutar
make run

# Ejecutar pruebas
make test

# Limpiar archivos generados
make clean

# Recompilar desde cero
make rebuild

# Ver información de compilación
make info

# Ver ayuda
make help
```

Uso

Iniciar el Programa

```
Make run
```

Comandos Disponibles

CREATE - Crear Archivo

```
CREATE <nombre> <tamaño>
```

Crea un archivo con el nombre y tamaño especificados.

Ejemplo:

```
> CREATE archivo1.txt 1000
Archivo 'archivo1.txt' creado exitosamente (1000 bytes)
```

WRITE - Escribir Datos

```
WRITE <nombre> <offset> <datos>
```

Escribe datos en el archivo desde la posición especificada.

Ejemplo:

```
> WRITE archivo1.txt 0 "Hola, mundo"  
Datos escritos exitosamente en 'archivo1.txt'
```

READ - Leer Datos

```
READ <nombre> <offset> <tamaño>
```

Lee una cantidad específica de bytes desde una posición del archivo.

Ejemplo:

```
> READ archivo1.txt 0 11  
Salida: "Hola, mundo"
```

DELETE - Eliminar Archivo

```
DELETE <nombre>
```

Elimina el archivo del sistema y libera sus bloques.

Ejemplo:

```
> DELETE archivo1.txt  
Archivo 'archivo1.txt' eliminado exitosamente
```

LIST - Listar Archivos

```
LIST
```

Muestra todos los archivos en el sistema con su tamaño.

Ejemplo:

```
> LIST
Archivos en el sistema:
Nombre                                Tamaño
-----
archivo1.txt                         1000 bytes
documento.txt                        2048 bytes
```

Ejemplos de Uso Completo

Ejemplo 1: Flujo Básico

```
> CREATE archivo1.txt 1000
Archivo 'archivo1.txt' creado exitosamente (1000 bytes)

> WRITE archivo1.txt 0 "Hola, mundo"
Datos escritos exitosamente en 'archivo1.txt'

> READ archivo1.txt 0 11
Salida: "Hola, mundo"

> LIST
Archivos en el sistema:
Nombre                                Tamaño
-----
archivo1.txt                         1000 bytes

> DELETE archivo1.txt
Archivo 'archivo1.txt' eliminado exitosamente

> LIST
(no hay archivos)
```

Ejemplo 2: Múltiples Archivos

```
> CREATE doc1.txt 500
> CREATE doc2.txt 1000
> CREATE doc3.txt 1500

> WRITE doc1.txt 0 "Primera línea"
> WRITE doc2.txt 0 "Segunda línea"
> WRITE doc3.txt 0 "Tercera línea"
```

```
> LIST
Archivos en el sistema:
Nombre                                Tamaño
-----
doc1.txt                             500 bytes
doc2.txt                             1000 bytes
doc3.txt                             1500 bytes
```

Ejemplo 3: Escritura con Offset

```
> CREATE datos.txt 100
> WRITE datos.txt 0 "Inicio"
> WRITE datos.txt 50 "Medio"
> READ datos.txt 0 6
Salida: "Inicio"
> READ datos.txt 50 5
Salida: "Medio"
```

Manejo de Errores

El sistema maneja diversos tipos de errores:

- **Archivo ya existe:** Intento de crear un archivo duplicado
- **Archivo no encontrado:** Operación sobre archivo inexistente
- **No hay espacio:** Almacenamiento insuficiente
- **Offset inválido:** Lectura/escritura fuera de límites
- **Tamaño inválido:** Tamaño de archivo no válido
- **Límite de archivos:** Máximo de archivos alcanzado

Diseño e Implementación

Patrones de Diseño Utilizados

1. **Separación de Responsabilidades:** Cada módulo tiene una función específica
2. **Encapsulamiento:** Estructuras de datos ocultas tras APIs limpias
3. **Manejo Centralizado de Errores:** Sistema unificado de códigos de error
4. **Gestión de Recursos:** Liberación automática de bloques al eliminar archivos

Decisiones de Diseño

1. **Bloques de tamaño fijo (512B):** Simplifica la gestión y es estándar en sistemas reales
2. **Asignación dinámica de bloques:** Eficiencia en uso de memoria
3. **Tabla de archivos estática:** Optimización para búsquedas rápidas
4. **Validación exhaustiva:** Robustez ante entradas inválidas

Gestión de Bloques

- Los bloques se asignan dinámicamente según demanda

- Bitmap de bloques libres/ocupados para rápida asignación
- Los bloques pueden ser no contiguos (fragmentación permitida)
- Liberación automática al eliminar archivos

Pruebas

El script `test_script.sh` ejecuta 8 pruebas automáticas:

1. Crear archivo
2. Escribir y leer datos
3. Eliminar archivo
4. Manejo de archivo duplicado
5. Lectura de archivo inexistente
6. Múltiples archivos simultáneos
7. Escritura con offset
8. Validación de tamaño inválido

Ejecutar pruebas:

```
make test
```

Estructura de Datos

File Entry (Entrada de Archivo)

```
typedef struct {  
    char name[MAX_FILENAME];           // Nombre del archivo  
    uint32_t size;                      // Tamaño en bytes  
    uint32_t block_count;               // Número de bloques  
    uint32_t blocks[MAX_BLOCKS];       // Índices de bloques  
    bool in_use;                        // Estado del slot  
} file_entry_t;
```

Block System (Sistema de Bloques)

```
typedef struct {  
    uint8_t data[MAX_BLOCKS][BLOCK_SIZE]; // Datos  
    bool allocated[MAX_BLOCKS];            // Mapa de asignación  
    uint32_t free_blocks;                  // Contador de libres  
} block_system_t;
```